Running an LLM Locally

Assignment Report: Local Model Setup, Python Integration and Temperature Testing

Course	Artificial Intelligence (CSC425)
Submitted to	Kh Ehsanur Rahman
Student name	S M Mashraful Islam
Student ID	2230836
Program	Computer Science (undergraduate)
University	Independent University Bangladesh (IUB)
Submission date	14 October 2026

Table of Contents

1. Introduction **3**

2. Hardware and software environment **3**

3. Part 1 — Local model setup **4**

3.1 Installing and verifying Ollama 4

3.2 Downloading and listing the model 4

3.3 Running an interactive conversation 5

3.4 Verifying local inference 5

3.5 Testing with the internet disconnected 6

3.6 Part 1 summary 6

4. Part 2 — Python integration **7**

4.1 Project structure 7

4.2 Configuration and file-based inputs 8

4.3 Explanation of the Python workflow 8

4.4 Execution and CSV output 9

5. Configuration adjustment: maximum output tokens **10**

6. Part 3 — Model testing and results **10**

6.1 Task 1: Capability summary 11

6.2 Task 2: Text summarization 11

6.3 Temperature comparison 14

6.4 Task 3: Sentiment classification 15

6.5 Task 4: Professional email 15

7. Overall discussion **16**

8. Conclusion **17**

Appendix **17**

A. Performance statistics by task 17

B. Screenshot inventory 17

1. Introduction

Large language models (LLMs) are usually used through online services. Running a model locally keeps prompts and responses on the user's own computer, allows the model to work without an internet connection once it has been downloaded, and avoids dependence on a remote provider. The trade-off is that speed depends on the local hardware.

Ollama is a tool that downloads, manages and serves language models on a personal computer. It offers a command-line interface for interactive use and a local HTTP API at `http://localhost:11434` that programs can call. In this assignment, Ollama runs the `gemma4:e2b` model.

A Python program makes the experiments repeatable. Because the configuration, system instructions, input texts and prompts are read from files, the same experiment can be run again without editing the code, and every model call is saved to a CSV file for later analysis. Temperature controls how much randomness the model uses when it chooses each next word: low values favour the most likely words, while higher values allow more varied wording. The summarization task is therefore repeated at temperatures 0.0, 0.3 and 1.0, twice at each, so that consistency, format compliance and accuracy can be compared.

This report documents the local setup (Part 1, Section 3), the Python program (Part 2, Section 4), a deviation from the standard configuration (Section 5), and the model outputs with their evaluation (Part 3, Section 6). It ends with a discussion, a conclusion and two short appendices.

2. Hardware and software environment

The values below are taken from the screenshots, the source files and the device specifications. Values that could not be confirmed from the evidence are marked as such.

Item	Details
Operating system	Windows (edition and version not shown in the screenshots). Commands were run in Git Bash (MINGW64).
Processor	Intel Core i3 (specific model not stated)
RAM	12 GB
GPU	113 MB (as reported; no dedicated GPU is required). `ollama ps` reports **100% CPU** for the loaded model (Figure 4), so inference ran on the CPU.
Storage	1.03 TB (as reported)
Ollama version	0.40.2 (Figure 1)
Model	`gemma4:e2b` (ID b37049369adf; 4.6 GB on disk, Figure 2)
Runtime details	Runner llamacpp; context length 4096; loaded size 970 MB (Figure 4)
Programming language	Python (version not shown); the `requests` library is used for HTTP calls
Interface	Ollama local REST API: `POST http://localhost:11434/api/chat`

A dedicated GPU is not strictly necessary for local inference: the assignment states that the model runs without a graphics card, but responses are slower on a CPU-only machine. Performance depends on the processor, the amount of RAM and whether a GPU is available. In this experiment the model ran on the CPU, and the six summarization calls took between 54 and 82 seconds each (Section 6.3).

The table compares the computer with the hardware requirements stated in the assignment.

Component	Your device	Assignment requirement
RAM	12 GB	8 GB minimum, 16 GB recommended (minimum met; below the recommended amount)
CPU	Intel Core i3	No specific CPU requirement
GPU	113 MB	Dedicated GPU not required
Storage	1.03 TB	About 7 GB free space required

3. Part 1 — Local model setup

This part documents each setup step with the command used, its purpose, the result shown in the screenshot, and the corresponding figure.

3.1 Installing and verifying Ollama

```
ollama --version
```

This command checks whether Ollama is installed and prints the installed version. The terminal in Figure 1 prints `ollama version is 0.40.2`, confirming that Ollama is installed and can be called from the command line.

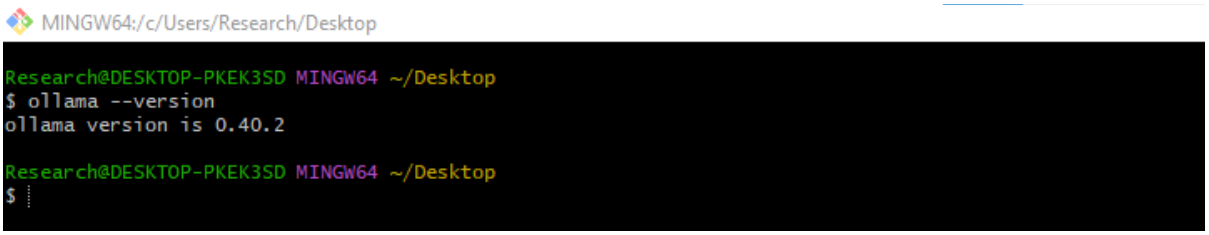


Figure 1: Ollama version verification (ollama --version).

3.2 Downloading and listing the model

```
ollama pull gemma4:e2b
ollama list
```

The first command downloads the model to the computer (the assignment notes that about 7 GB of free disk space is needed). The second lists the models stored locally. Figure 2 shows the `ollama list` output: `gemma4:e2b`, ID `b37049369adf`, size 4.6 GB, modified 7 minutes earlier. The download progress of `ollama pull` itself was not captured in a screenshot; Figure 2 shows that the model is present locally.

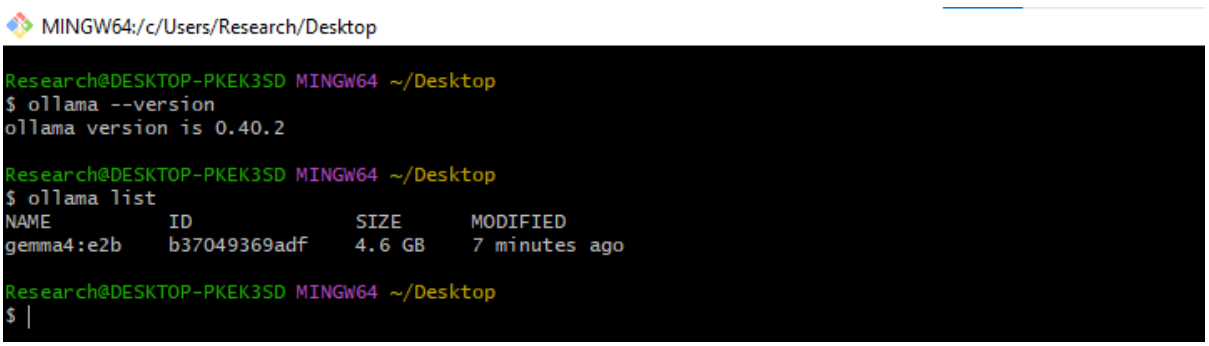


Figure 2: Model listing (ollama list) showing gemma4:e2b.

3.3 Running an interactive conversation

```
ollama run gemma4:e2b
```

This command opens an interactive session in which the user types a question and the locally installed model replies; typing `/bye` ends the session. Figure 3 shows the end of such a session. The typed question has scrolled out of view, but the model's visible reasoning states that the user wants an explanation of “what a database is” in exactly two sentences. After the line “...done thinking.” the model gives a two-sentence answer: “A database is an organized collection of structured data, typically stored electronically, designed to efficiently manage and store large volumes of information. It uses a structured system to allow users and applications to quickly retrieve, update, and analyze the data they need.” The session is then closed with `/bye`.

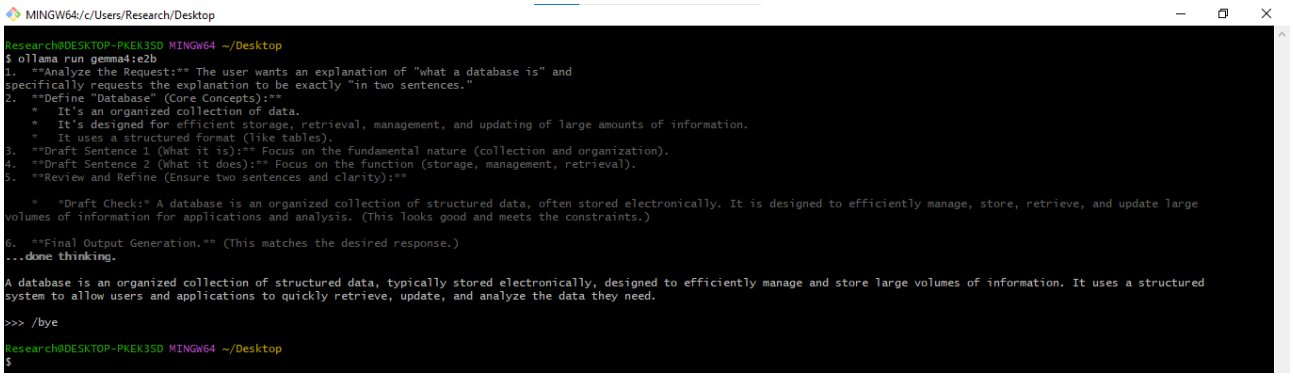


Figure 3: Interactive session with ollama run gemma4:e2b, ended with /bye.

3.4 Verifying local inference

```
ollama ps
```

This command lists the models that Ollama currently has loaded in memory on this computer. Figure 4 shows two consecutive runs of the command. Both list `gemma4:e2b` with a loaded size of 970 MB, processor **100% CPU**, context length 4096 and runner `llamacpp`. Because the model is listed by the local Ollama service and its processor is the local CPU, the output supports the claim that the model is executed on this machine. The UNTIL column falls from “4 minutes from now” to “3 minutes from now”; this is the countdown before an idle model is unloaded, which shows that the model was resident after recent use. The capture was taken while the model was loaded, but not at the moment a response was being generated.

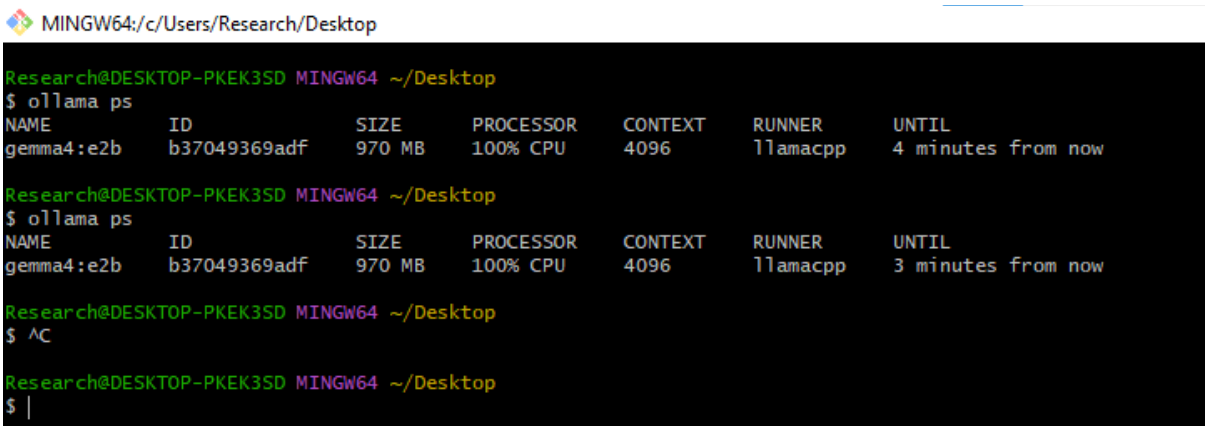


Figure 4: Loaded model shown by ollama ps (100% CPU, runner llamacpp).

3.5 Testing with the internet disconnected

The model was tested with the internet disconnected to demonstrate that the downloaded model can generate answers without an active connection. Figure 5 shows a session in which the question “what are the skill set to learn for mastering agentic ai?” has been entered after an earlier, longer answer about AI agents, and the model is in its “Thinking...” stage. The network icon in the Windows system tray is circled in red; Figure 6 shows that region enlarged. The icon is a globe with a prohibited-sign badge, which Windows displays when no internet connection is available.

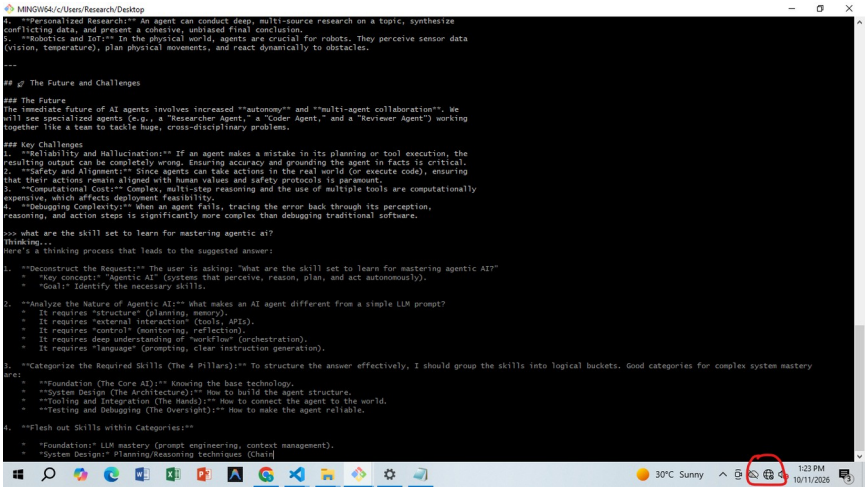


Figure 5: Model generating a response with the network icon circled in the system tray.



Figure 6: Enlarged crop of the system-tray area of Figure 5 (red circle is part of the original screenshot).

Limitation: the screenshot was taken while the answer to the new question was still being generated, so the completed answer to that question is not shown. It therefore provides partial evidence of the offline test; a capture showing both the disconnected icon and a finished answer would make the evidence complete.

3.6 Part 1 summary

Requirement	Evidence	Status
Ollama installed, version shown	Figure 1: `ollama version is 0.40.2`	Verified
Model downloaded with `ollama pull gemma4:e2b`	No screenshot of the pull command; Figure 2 shows the model present locally	Evidence not supplied
Model listed with `ollama list`	Figure 2: gemma4:e2b, 4.6 GB	Verified
Interactive chat with `ollama run gemma4:e2b`	Figure 3: model reply and `/bye` (question scrolled out of view)	Verified
`ollama ps` during a query	Figure 4: model loaded, 100% CPU; not captured mid-generation	Partially verified
Answer produced with internet disconnected	Figures 5 and 6: response in progress with no-connection tray icon; finished answer not captured	Partially verified
Hardware recorded (processor, RAM, GPU)	Section 2 lists processor, RAM, GPU and storage; CPU-only execution shown in Figure 4	Verified

4. Part 2 — Python integration

4.1 Project structure

The project follows the folder layout required by the assignment. The program source (`run_llm.py`, `analyze_results.py`) and the contents of `results.csv` were reviewed for this report.

```
project/
|-- run_llm.py
|-- analyze_results.py
|-- config.json
|-- system/
|   `-- default.txt
|-- inputs/
|   |-- passage.txt
|   `-- review.txt
|-- prompts/
|   |-- capabilities.txt
|   |-- summarize.txt
|   |-- sentiment.txt
|   `-- email.txt
`-- outputs/
    |-- results.csv
    |-- task_summary.csv
    `-- temperature_summary.csv
```

Path	Role	Basis in the evidence
`run_llm.py`	Main program: runs the four tasks and saves the CSV	Source code reviewed
`analyze_results.py`	Reads results.csv and writes the two summary CSV files	Source code reviewed
`config.json` †	Model, temperatures, runs, default temperature, max tokens	Read by run_llm.py
`system/default.txt` †	System instructions	Read by run_llm.py
`inputs/passage.txt`, `inputs/review.txt` †	Source passage and customer review	Read by run_llm.py
`prompts/*.txt` (4 files) †	Task prompts with {input} placeholders	Read by run_llm.py
`outputs/results.csv`	One row per model call	Contents reviewed (9 rows)
`outputs/task_summary.csv`, `outputs/temperature_summary.csv` †	Summary statistics	Produced by analyze_results.py

† These files are expected in the submitted project folder. They were not available as separate files when this report was prepared, so their contents are not reproduced here; the standard versions are those given in the assignment appendix, and the summary statistics in this report were recalculated directly from `results.csv`.

4.2 Configuration and file-based inputs

The program reads its settings from `config.json`: the model name, the list of temperatures, the number of runs per setting, the default temperature and the maximum number of output tokens. The standard configuration specified by the assignment is shown below; the value of `max_tokens` that was actually used is discussed in Section 5.

```
{
  "model": "gemma4:e2b",
  "temperatures": [0.0, 0.3, 1.0],
  "runs_per_setting": 2,
  "default_temperature": 0.0,
  "max_tokens": 512
}
```

Directory / file	Role
`system/`	System-level instructions, sent to the model as the system message
`inputs/`	Source texts: the renewable-energy passage and the customer review
`prompts/`	Task-specific prompts; the marker {input} shows where an input text is inserted
`outputs/`	Recorded results written by the program

The program reads these files at run time. Reviewing `run_llm.py` confirms that it contains file paths, the API address and the program logic, but none of the instruction, prompt or input text. Changing a text file therefore changes the experiment without editing any Python code. The assignment requires the system instructions, inputs and prompts to remain unchanged from its appendix.

4.3 Explanation of the Python workflow

The table lists each requirement and how the source code of `run_llm.py` implements it. Each statement was checked against the code.

#	Requirement	Implementation in run_llm.py	Status
1	Read settings from config.json	`main()` loads the file with `read_text_file` and `json.loads`, then takes model, temperatures, runs_per_setting, default_temperature and max_tokens	Implemented
2	Read instructions, prompts and inputs from files	`system/default.txt`, the four prompt files, `passage.txt` and `review.txt` are read as UTF-8 text	Implemented
3	Send system instructions as the system message	`call_model` builds a `messages` list with a “system” message followed by a “user” message	Implemented
4	Insert input text at {input}	`str.replace("{input}", ...)` is applied to the summarization prompt (passage) and the sentiment prompt (review); the capabilities and email prompts have no placeholder	Implemented
5	Call the local Ollama API	POST to `http://localhost:11434/api/chat` with `stream` set to false; `temperature` and `num_predict` (from max_tokens) are passed as options; timeout 600 s	Implemented
6	Summarize at every temperature, repeated runs_per_setting times	Nested loops over `temperatures` and `range(1, runs_per_setting + 1)` give six summarization calls	Implemented

#	Requirement	Implementation in run_llm.py	Status
7	Run other tasks once at default temperature	Loop over capabilities, sentiment and email with `default_temperature` and run number 1	Implemented
8	Record task, temperature, run, time, word count, response	Each call is stored with `time.perf_counter` timing (3 decimals), `len(answer.split())` as word count, and the full response	Implemented
9	Print labelled responses	Prints "=== Summarization | Temperature t | Run n ===" before each summary and "=== Task ===" before the others	Implemented
10	Save every call to outputs/results.csv	Writes the file with `csv.DictWriter` (UTF-8 with BOM), creating the folder if needed	Implemented
11	Clear errors for missing files or unavailable Ollama	A missing file raises "Missing file: path"; before any model call the program requests `/api/tags` and, on failure, reports that Ollama is not running or cannot be reached; invalid JSON, failed requests and missing keys are also caught. All errors print "ERROR: ..." and exit with status 1	Implemented

Observations from the source code that affect how the results should be read:

- The word count is the number of whitespace-separated tokens, so bullet and Markdown symbols such as `*` and `-` are counted as words. Each saved summary therefore includes three extra "words".
- The CSV is written once, after all nine calls have finished. A failure part-way through leaves no partial file, and every row of a given `results.csv` comes from a single execution and configuration.
- The Ollama availability check runs after the text files are loaded, so a missing text file is reported before an unavailable service. A missing configuration key is reported only by its name, and a model that has not been downloaded would appear through the general request-failure message.

4.4 Execution and CSV output

```
python run_llm.py
python analyze_results.py
```

The first command runs all nine model calls and writes `outputs/results.csv` with the columns `task`, `temperature`, `run_number`, `response_time_seconds`, `word_count` and `response`. The second reads that file and writes `task_summary.csv` and `temperature_summary.csv`.

The reviewed `results.csv` contains nine data rows, which matches the experiment design: six summarization runs and one run each of the other three tasks. For all nine rows, the recorded word count equals a recount of the saved response.

Task	Temperature(s)	Runs each	Calls in results.csv
summarize	0.0, 0.3, 1.0	2	6
capabilities	0.0	1	1
sentiment	0.0	1	1
email	0.0	1	1
Total			**9**

5. Configuration adjustment: maximum output tokens

The standard configuration specifies `max_tokens: 512`. This section documents a deviation from that value.

Setting	Assignment standard	Final run (results.csv)
max_tokens	512	**1024** [confirm against the submitted config.json]
model, temperatures, runs_per_setting, default_temperature	gemma4:e2b; [0.0, 0.3, 1.0]; 2; 0.0	Consistent with the standard values: the CSV contains two summarization runs at each of 0.0, 0.3 and 1.0 and one run of each other task at 0.0

- The original assignment configuration specifies `max_tokens` of 512. The value 1024 was not required by the assignment.
- During testing, some generated responses appeared incomplete under the original setting. To investigate and reduce the chance of truncation, the maximum output-token setting was increased to 1024. This was a practical troubleshooting adjustment.
- A larger limit permits longer responses, but it does not guarantee correctness, completeness or compliance with every prompt requirement. For example, the prompts limit length in words and bullets, which `max_tokens` does not enforce.
- The results in this report come from the final run with `max_tokens` set to 1024. The nine rows of `results.csv` were written by a single program execution, so they share one configuration and no results from different configurations are compared.
- In the final CSV, every response ends with a finished sentence or the requested sign-off, and no truncation is visible. This observation applies to this run only; the earlier incomplete outputs are not part of the submitted evidence.
- One possible contributing factor, which was not tested, is that the model prints a reasoning trace before its answer in interactive mode (Figures 3 and 5). If such tokens count towards the output limit, 512 tokens could be used up before the answer is finished.

Because `config.json` is one of the standard files that the assignment asks to be used as given, this change is disclosed rather than hidden. To align the evidence with the assignment, a separate validation run with the standard value of 512 is recommended. The program always writes to `outputs/results.csv`, so the final CSV should be copied under another name before any validation run so that it is not overwritten. The results and configuration in this report have not been altered.

6. Part 3 — Model testing and results

All model responses below are reproduced verbatim from `outputs/results.csv`, with line breaks and Markdown symbols preserved. Requirements are taken from the assignment prompts. Word counts of bullets exclude the leading bullet symbol; the word counts recorded in the CSV include it.

6.1 Task 1: Capability summary

Task: capabilities | **Temperature:** 0.0 | **Run:** 1 | **Response time:** 64.406 s | **Recorded word count:** 78

Capabilities:
- Answering complex questions across various subjects.
- Generating creative text formats, such as stories and code.
- Summarizing long documents and extracting key information.
- Translating text between multiple languages.

Limitations:
- Knowledge is based on training data and may have a cutoff date.
- Cannot perform physical actions or interact with the real world.
- Output accuracy depends on the quality and clarity of the input provided.
- May occasionally generate inaccurate or nonsensical information.

Requirement	Observation	Status
“Capabilities:” section present	Heading “Capabilities:” followed by 4 hyphen bullets	Met
“Limitations:” section present	Heading “Limitations:” followed by 4 hyphen bullets	Met
3 to 5 bullets in each section	4 bullets under each heading	Met
Whole answer under 150 words	70 words excluding the 8 bullet symbols (78 tokens as recorded in the CSV)	Met
Exact format, no extra text	No introduction or closing remarks; blank line between the sections	Met

The response follows the required format. Its statements are general and accurate for a language model (for example, knowledge limited by training data and no physical actions), and no formatting or content issue was found.

6.2 Task 2: Text summarization

The six summarization outputs are shown in the order temperature 0.0 (runs 1 and 2), 0.3 (runs 1 and 2) and 1.0 (runs 1 and 2). The reference is the standard renewable-energy passage from the assignment appendix.

Temperature 0.0 — Run 1

Response time: 81.716 s | **Recorded word count:** 52

* Renewable energy is central due to falling solar costs and cheap onshore wind generation worldwide.
* Obstacles include the need for grid storage, high financing costs for developing countries, and concentrated supply chains.
* Governments are using tools like auctions and carbon pricing, but coordination and infrastructure are the main concerns.

Temperature 0.0 — Run 2

Response time: 63.678 s | **Recorded word count:** 47

* Renewable energy is central due to falling solar costs and cheap onshore wind generation worldwide.
* Challenges include grid stability needs, expensive long-term storage, and financing difficulties for developing countries.
* Closing gaps in money, infrastructure, and international coordination will determine if emissions targets are met.

Temperature 0.3 — Run 1

Response time: 54.772 s | **Recorded word count:** 42

* Renewable energy is central due to falling solar costs and cheap onshore wind power.
* Transition obstacles include grid needs for storage, high financing costs, and concentrated supply chains.
* Success depends on addressing money, infrastructure, and international coordination between countries.

Temperature 0.3 — Run 2

Response time: 65.058 s | **Recorded word count:** 52

* Renewable energy has become central due to falling solar costs and the low cost of onshore wind power.
* Transition obstacles involve grid stability needs, expensive long-term storage, and financing difficulties for developing nations.
* Concerns exist regarding concentrated supply chains and the need for better coordination on money and infrastructure.

Temperature 1.0 — Run 1

Response time: 62.607 s | **Recorded word count:** 48

* Renewable energy is central due to falling solar costs and cheap onshore wind generation globally.
* Transition obstacles include the need for energy storage and facing high financing costs in developing countries.
* Success depends on addressing issues related to money, infrastructure, and international coordination between countries.

Temperature 1.0 — Run 2

Response time: 54.189 s | **Recorded word count:** 51

* Renewable energy is central due to falling solar costs and cheap onshore wind, leading to increased new capacity.
* Obstacles include the need for storage, high financing costs for developing nations, and concentrated supply chains.
* Future success depends on resolving gaps related to money, infrastructure, and international coordination efforts.

Format compliance

Output	Bullets	One sentence each	Words per bullet	All bullets ≤ 25 words
Temp 0.0, run 1	3	Yes	15, 17, 17	Yes
Temp 0.0, run 2	3	Yes	15, 14, 15	Yes
Temp 0.3, run 1	3	Yes	14, 14, 11	Yes
Temp 0.3, run 2	3	Yes	18, 15, 16	Yes
Temp 1.0, run 1	3	Yes	15, 16, 14	Yes
Temp 1.0, run 2	3	Yes	18, 16, 14	Yes

Coverage of the passage

The passage contains six main ideas. A three-bullet summary cannot include all of them, so omissions are expected; the table shows which ideas each summary kept.

Main idea	0.0 R1	0.0 R2	0.3 R1	0.3 R2	1.0 R1	1.0 R2
Solar costs have fallen; onshore wind is cheap	Yes	Yes	Yes	Yes	Yes	Yes
Variable output requires grid storage and flexibility	Yes	Yes	Yes	Yes	Yes	Yes
Developing countries face high financing costs	Yes	Yes	Partly	Yes	Yes	Yes
Supply chains are concentrated	Yes	No	Yes	Yes	No	Yes
Governments are responding with policy tools	Yes	No	No	No	No	No
Money, infrastructure and coordination decide whether targets are met	Partly	Yes	Yes	Partly	Yes	Yes
Ideas covered (full / partly)	5 / 1	4 / 0	4 / 1	4 / 1	4 / 0	5 / 0

Accuracy checks against the passage:

- All six summaries state the central claims correctly (falling solar costs and cheap onshore wind, the need for storage and grid support, and financing difficulties in developing countries). None adds a fact that is absent from the passage.
- Some wording generalises slightly. “Worldwide” (temperature 0.0, runs 1 and 2) and “globally” (temperature 1.0, run 1) extend the passage's statement that onshore wind is the cheapest new electricity “in many countries”. Temperature 0.0 run 2 says that closing the gaps “will determine” whether targets are met, whereas the passage says it will “largely determine” this.
- Three summaries (0.3 run 1, 1.0 run 1, 1.0 run 2) say that “success” depends on money, infrastructure and coordination without saying what success means; only temperature 0.0 run 2 names emissions targets.
- The final bullet of temperature 0.3 run 2 (“better coordination on money and infrastructure”) is the least clear, because it blends two separate points. Temperature 1.0 run 2 says renewables lead to “increased new capacity”, which is supported by the passage's statement that most new capacity is renewable.

6.3 Temperature comparison

Temp.	Runs	Avg. time (s)	Avg. words	Consistency and quality observations
0.0	2	72.70	49.5	Both runs share an identical first bullet; bullets 2 and 3 differ. Run 1 covers five ideas fully (including supply chains and government tools). Run 2 omits those two but adds the cost of long-term storage and mentions emissions targets. Both fully compliant.
0.3	2	59.92	47.0	No identical bullets between the two runs. Both mention supply chains and omit government tools. Run 1 is the shortest output (42 recorded words). The last bullet of run 2 is unclear. Both fully compliant.
1.0	2	58.40	49.5	No identical bullets between the two runs. Run 1 omits supply chains; run 2 includes them and covers five ideas. Wording is no more creative or less accurate than at the lower temperatures. Both fully compliant.

Averages are calculated from the six summarization rows of `results.csv`. The recorded word counts include the three bullet symbols in each response.

- **Similarities.** Five of the six summaries open with the same clause (“Renewable energy is central due to falling solar costs and cheap onshore wind”); the sixth (0.3 run 2) says “has become central”. All six cover the cost fall, storage and grid needs, and financing difficulties, and all close with the money, infrastructure and coordination question or with government action.
- **Differences.** Differences are in wording and in which two secondary ideas fill bullets 2 and 3. At temperature 0.0 the first bullet is identical in the two runs, whereas at 0.3 and 1.0 no bullet is identical. Even at temperature 0.0 the two runs were therefore not word-for-word identical.
- **Format.** All six responses use exactly three bullets of one sentence each, and the longest bullet has 18 words, well within the 25-word limit.
- **Omissions and unsupported claims.** Every summary omits some ideas, as expected for three bullets. Temperature 0.0 run 2 and temperature 1.0 run 1 omit supply chains, and only temperature 0.0 run 1 mentions government policy tools. No clearly unsupported claim appears; the minor generalisations are listed in Section 6.2.
- **Response time.** Average times were 72.70 s, 59.92 s and 58.40 s for temperatures 0.0, 0.3 and 1.0. The 0.0 average is raised by the slowest call in the whole run (81.716 s), which was the first call; the CSV does not show whether this includes one-off model loading. These differences are observations from this experiment only and do not show that temperature changes speed.
- **Best temperature for this task.** Temperature 0.0 is recommended. All six outputs satisfy the format, so the choice rests on accuracy and consistency: temperature 0.0 produced the most complete single summary (run 1) and the most precise statement of the final point (run 2), and a factual summarization task benefits from the setting that reduces random variation. However, the evidence for this is modest: coverage does not clearly separate the temperatures (temperature 1.0 run 2 covers as many ideas as 0.0 run 1), and the two runs at 0.0 still differed.
- **Limitation.** Only two runs were performed at each temperature. This small sample cannot establish a general relationship between temperature and quality or response time.

6.4 Task 3: Sentiment classification

Task: sentiment | **Temperature:** 0.0 | **Run:** 1 | **Response time:** 14.654 s | **Recorded word count:** 39

Label: Mixed
Explanation: The reviewer praises the excellent sound quality and long battery life, but also notes issues with uncomfortable ear cushions and frequent app disconnections. Additionally, the slow response time from customer support contributes to the mixed sentiment.

Requirement	Observation	Status
Label is exactly Positive, Negative or Mixed	Label: Mixed	Met
Uses the “Label:” and “Explanation:” format	Both lines present, in that order	Met
Explanation has exactly two sentences	Two sentences (the second begins “Additionally,”)	Met
Cites specific points from the review	Excellent sound quality, long battery life, uncomfortable ear cushions, app disconnections, slow customer support	Met
Label is correct	The review praises sound quality and two-day battery life but criticises comfort, app stability and a week-long support delay, so Mixed is correct	Met

The response is correct and well formed. The first sentence is long because it covers four points, and the second covers only the support delay; the explanation paraphrases the review (“slow response time”) rather than quoting its details such as “two full days” or “a week”.

6.5 Task 4: Professional email

Task: email | **Temperature:** 0.0 | **Run:** 1 | **Response time:** 72.420 s | **Recorded word count:** 109

Subject: Meeting Request: Project Progress Update

Dear [Supervisor's Name],

I hope this email finds you well.

I am writing to request a 30-minute meeting next week to discuss the progress of my project. I am pleased to inform you that data collection is complete, and the analysis phase is currently underway. I would appreciate the opportunity to review the initial findings and discuss the next steps.

Would you be available on Tuesday at 10:00 AM or Thursday at 2:00 PM? Please let me know if either of these times works for you, or if another time is more convenient.

Thank you for your time and guidance.

Sincerely,

Student Name

Requirement	Evidence from the output	Status
Clear subject line	Subject: “Meeting Request: Project Progress Update”	Met
Requests a 30-minute meeting next week	“request a 30-minute meeting next week”	Met
States data collection is complete	“data collection is complete”	Met
States analysis is underway	“the analysis phase is currently underway”	Met
Offers two time slots	Tuesday at 10:00 AM; Thursday at 2:00 PM	Met
Polite, formal tone	Formal greeting, “I hope this email finds you well”, “Thank you for your time and guidance”	Met
Under 150 words	109 tokens recorded in the CSV, including the subject line and placeholder	Met
Signs off as “Student Name”	Final line is “Student Name”, after the closing “Sincerely,”	Met

All stated requirements are met. Two minor points: the salutation keeps the placeholder “[Supervisor's Name]”, which is reasonable because no name was supplied, and the saved subject line contains literal Markdown asterisks around “Subject:”. The offer to “review the initial findings” is an elaboration, not a factual claim about the project.

7. Overall discussion

Verified in this experiment

- Ollama 0.40.2 lists `gemma4:e2b` as a local model and the model answers interactively (Figures 2 and 3); `ollama ps` shows it loaded and running on the CPU (Figure 4).
- The Python program read its configuration, instructions, prompts and inputs from files and produced nine recorded model calls in one execution, matching the experiment design (Section 4).
- The CSV records the task, temperature, run number, response time and word count for every call, so response times and lengths could be compared (Section 6.3).
- All six summaries followed the three-bullet, one-sentence, 25-word format at every temperature. Wording differed between runs at every temperature, including 0.0, but the observed differences were small and did not introduce unsupported claims.
- The capability, sentiment and email outputs met all of their stated format requirements (Sections 6.1, 6.4 and 6.5).
- The saved responses are complete, but they were produced with `max_tokens` of 1024 rather than the standard 512, which is why the setting is documented (Section 5).

General explanations (not tested here)

- Temperature controls sampling randomness. Lower values are generally preferred for factual tasks where consistency matters, and higher values for tasks where variety is useful.
- An output-token limit caps the length of the generated text; it cannot enforce word or bullet limits, so prompt wording and checking of the output remain necessary.
- Local execution gives privacy and offline use, but speed depends on the hardware; on a CPU-only computer each call can take about a minute.
- Small models can produce fluent text that is subtly inaccurate, so outputs should be checked against the source text, as was done in Section 6.2.

8. Conclusion

The `gemma4:e2b` model was installed through Ollama 0.40.2 and used locally. Screenshots verify the version, the model listing, an interactive chat and a loaded model running on the CPU. The offline test and the `ollama ps` evidence are partial, and the model download step is not documented by a screenshot.

A Python program reads all settings, instructions, prompts and inputs from files, calls the local API, prints labelled responses, handles missing files and an unavailable service, and saved nine model calls to `results.csv`. The capability, sentiment and email responses met all stated requirements. All six summaries followed the required format and stayed close to the passage, with only minor generalisations and different choices of which ideas to include.

For the summarization task, temperature 0.0 is recommended, mainly because a factual task benefits from the least random setting and because its outputs were accurate and fully compliant. The evidence for preferring it is modest: only two runs were made at each temperature, the temperatures were not clearly separated by quality, and the two runs at 0.0 were not identical. The results were generated with `max_tokens` increased from 512 to 1024 after some outputs appeared incomplete, so a validation run with the standard value is recommended.

Appendix

A. Performance statistics by task

These values are recalculated from `results.csv` using the same calculations as `analyze_results.py` (averages of response time and word count, grouped by task).

Task	Calls	Avg. time (s)	Fastest (s)	Slowest (s)	Avg. words
capabilities	1	64.41	64.41	64.41	78.00
email	1	72.42	72.42	72.42	109.00
sentiment	1	14.65	14.65	14.65	39.00
summarize	6	63.67	54.19	81.72	48.67
All calls	**9**	**59.28**	**14.65**	**81.72**	

The sentiment call was by far the fastest (14.65 s), consistent with its short output; the other calls took roughly 54–82 s each on the CPU-only computer.

B. Screenshot inventory

Figure	Content	Source screenshot file	Clock shown
1	ollama --version	Screenshot_2026-10-11_101932.png	10:19 AM
2	ollama list	Screenshot_2026-10-11_102023.png	10:20 AM
3	ollama run gemma4:e2b	Screenshot_2026-10-11_102640.png	10:26 AM
4	ollama ps (cropped)	Screenshot_2026-10-11_last.png	2:47 PM
5, 6	Response in progress, network icon (Figure 6 is an enlarged crop)	Screenshot_2026-10-11_132403.png	1:23 PM

Figures 1–4 are cropped to the terminal area for readability and Figure 6 is an enlarged crop; no screenshot content was altered.